



OPTIMIZATION REPORT



Tran Ngoc Hang
Le Bao Khanh
Luong Quynh Chi
Le Phuong Thao
Nguyen Duc Anh
Nguyen Thi Huong Tra

OPTIMIZATION MIDTERM REPORT

Group of authors

Tran Ngoc Hang (Leader)

Le Bao Khanh

Luong Quynh Chi

Le Phuong Thao

Nguyen Duc Anh

Nguyen Thi Huong Tra

Problem 1

Minimum fuel optimal control

We consider a linear dynamical system with state $x(t) \in \mathbb{R}^n$, $t = 0, \dots, N$, and actuator or input signal $u(t) \in \mathbb{R}$, for $t = 0, \dots, N - 1$. The dynamics of the system is given by the linear recurrence

$$x(t+1) = Ax(t) + bu(t), \quad t = 0, \dots, N-1,$$

where $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$ are given. We assume that the initial state is zero, i.e., $x(0) = 0$. The minimum fuel optimal control problem is to choose the input $u(0), \dots, u(N-1)$ to minimize the total fuel consumed, which is given by

$$F = \sum_{t=0}^{N-1} f(u(t))$$

subject to the constraint that $x(N) = x_{\text{des}}$, where N is the (given) time horizon, and $x_{\text{des}} \in \mathbb{R}^n$ is the (given) final or target state. The function $f : \mathbb{R} \rightarrow \mathbb{R}$ is the fuel use map for the actuator, which gives the amount of fuel used as a function of the actuator signal amplitude. In this problem we use

$$f(a) = \begin{cases} |a| & \text{if } |a| \leq 1 \\ 2|a| - 1 & \text{if } |a| > 1 \end{cases}$$

This means that fuel use is proportional to the absolute value of the actuator signal, for actuator signals between -1 and 1; for larger actuator signals the marginal fuel efficiency is half.

Questions:

- a) Formulate the minimum fuel optimal control problem as an LP.
- b) Solve the following instance of the problem:

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 0.95 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 0.1 \end{bmatrix}, \quad x(0) = (0, 0), \quad x_{\text{des}} = (10, 0), \quad N = 20.$$

We can interpret the system as a simple model of a vehicle moving in one dimension. The state dimension is $n = 2$, with $x_1(t)$ denoting the position of the vehicle at time t and $x_2(t)$ giving its velocity. The initial state is $(0, 0)$, which corresponds to the vehicle at rest at position 0; the final state is $x_{\text{des}} = (10, 0)$, which corresponds to the vehicle being at rest at position 10. Roughly speaking, this means that the actuator input affects the velocity, which in turn affects the position. The coefficient $A_{22} = 0.95$ means that velocity decays by 5 in one sample period if no actuator signal is applied.

- Solve the problem using **cvxpy** in two ways and check that the solutions match:
 - By keeping operations $|\cdot|$, max, min implicit and making **cvxpy** convert it to LP for you.
 - By applying directly the LP formulation you obtained in point 1.
- Plot the input signal $u(t)$ for $t = 0, \dots, 19$, and the position and velocity (i.e., $x_1(t)$ and $x_2(t)$) for $t = 0, \dots, 20$.

Solution.

a)

We have specified $x(0) = 0$ and $x(N) = x_{\text{des}}$. From the linear recurrence, we can imply:

$$\begin{aligned} x(1) &= bu(0) \\ x(2) &= Abu(0) + bu(1) \\ x(3) &= A^2bu(0) + Abu(1) + bu(2) \\ &\vdots \\ x(N) &= A^{N-1}bu(0) + A^{N-2}bu(1) + \dots + Abu(N-1) + bu(N-1) = x_{\text{des}} \end{aligned}$$

Now, let's define the controllability matrix:

$$C = [A^{N-1}b \ A^{N-2}b \ \dots \ Ab \ b]$$

Indeed, if we let $u^T = [u(0) \ \dots \ u(N-1)]$, then our initial and target conditions for the problem become $Cu = x_{\text{des}}$.

For the function $f(a)$ which represents the fuel use map for the actuator, we will try to formulate it to obtain an equivalent linear optimization. We can add new variables and inequalities to turn f into a linear objective.

Since it is a convex piecewise function, we can write f with the expression as follows:

$$f(a) = \begin{cases} |a| & \text{if } |a| \leq 1 \\ 2|a| - 1 & \text{if } |a| > 1 \end{cases} = \max\{|a|, 2|a| - 1\}$$

And let $t = \max\{|a|, 2|a| - 1\}$, then obtained the constraints:

$$\begin{aligned} |a| &\leq t \\ 2|a| - 1 &\leq t \end{aligned}$$

Rewriting above, we have the inequalities:

$$\begin{aligned} -t &\leq a \leq t \\ -\frac{t+1}{2} &\leq a \leq \frac{t+1}{2} \end{aligned}$$

Following this approach, we have added one new variable t_i and four new inequality constraints for each $u(i)$.

That means we can now introduce a vector $t = [t_0, \dots, t_{N-1}]$, one t_i for each $u(i)$. Hence, the total fuel consumed is given by the function:

$$F = \sum_{t=0}^{N-1} f(u(t))$$

now becomes:

$$F = \sum_{i=0}^{N-1} t_i$$

Our linear program is then:

$$\begin{aligned}
& \text{Minimize} && \mathbf{1}^T t \\
& \text{subject to} && -t \leq u \leq t \\
& && -\frac{t+1}{2} \leq u \leq \frac{t+1}{2} \\
& && Cu = x_{\text{des}}
\end{aligned}$$

b)

Firstly, we have to import **cvxpy** package as well as some necessary libraries for handling arrays and plotting data:

```
%matplotlib inline
import numpy as np
np.set_printoptions(precision=2, suppress=True)
import matplotlib.pyplot as plt
from cvxpy import *
```

For this problem, we are going to use the following given data:

- $N = 20$
- $n = 2$
- $A = \begin{bmatrix} 1 & 1 \\ 0 & 0.95 \end{bmatrix} \in \mathbb{R}^{2 \times 2}$
- $b = \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} \in \mathbb{R}^2$
- $x(0) = (0, 0) \in \mathbb{R}^2$
- $x_{\text{des}} = (10, 0) \in \mathbb{R}^2$

Now we input the given data into Python:

```
N = 20
n = 2
A = np.array([[1, 1], [0, 0.95]], dtype=float)
b = np.array([0, 0.1], dtype=float).reshape(n,1)
xdes = np.array([10, 0], dtype=float)
```

The first way of using **cvxpy** is keeping operations $|\cdot|$, max, min implicit and making **cvxpy** convert it to LP.

We are going to state the problem as a general optimization problem and let the **cvxpy** do the rest of the job.

$$\begin{aligned}
& \underset{x, u}{\text{minimize}} && \sum_{t=0}^{N-1} \max(|u_t|, 2|u_t| - 1) \\
& \text{subject to} && x(0) = 0 \\
& && x(t+1) = Ax(t) + bu(t), \text{ for } t = 0, \dots, N-1 \\
& && x(N) = x_{\text{des}}
\end{aligned}$$

Here is the code to solve the problem by using **cvxpy** package:

```

# Define the constraints and objective function coefficients
X = Variable([n, N+1])
u = Variable([1, N])
obj = Minimize(sum(maximum(abs(u), 2 * abs(u)-1)))
constr = [X[:, 0] == 0, X[:, 1:] == A @ X[:, :N] + b @ u, X[:, -1] == xdes]
# Solve the problem by using CVXPY
prob = Problem(obj, constr)
result = prob.solve()
print("Optimal value t*:", result)
print("Optimal control input u*:", u.value)

```

The result we obtained is:

```

Optimal value t*: 15.911948890496248
Optimal control input u*:
[[3.23  1.    1.    1.    1.    1.    1.    0.    0.
 0.    0.    0. -0.   -0.   -0.   -0.   -1.   -1.   -1. -1.23]]

```

The second way is applying directly the LP formulation you obtained in part a).

$$\begin{aligned}
 & \text{Minimize } \mathbf{1}^T t \\
 & \text{subject to } -t \leq u \leq t \\
 & \quad -\frac{t+1}{2} \leq u \leq \frac{t+1}{2} \\
 & \quad u = x_{des}
 \end{aligned}$$

where $t = [t_0, \dots, t_{N-1}]$, $u^T = [u(0) \ \dots \ u(N-1)]$, $C = [A^{N-1}b \ A^{N-2}b \ \dots \ Ab \ b]$

The Python code is represented below:

```

# Redefine the matrix A and vector b
A = np.array([[1, 1], [0, 0.95]], dtype=float)
b = np.array([0, 0.1], dtype=float).reshape(-1, 1)
# Reshape b to be a column vector

# Initialize C as an empty list
C = []

# Calculate and append the powers of A times b to C
for n in range(N, 0, -1):
    An = np.linalg.matrix_power(A, n - 1)
    C.append(np.dot(An, b))

# Stack the results horizontally to obtain the final C matrix
C = np.hstack(C)

# Define the constraints and objective function coefficients
U = Variable(N)
T = Variable(N)
objective = Minimize(np.ones(N).T @ T)
constraints = [
    -T <= U,
    U <= T,

```

```

-(T + np.ones(N))/2 <= U,
U <= (T + np.ones(N))/2,
C @ U == xdes
]

# Solve the problem by using CVXPY
prob = Problem(objective, constraints)
result = prob.solve()
print("Optimal value t*: ", np.sum(T.value))
print("Optimal control input u*: ", U.value)

The result we obtained is:

Optimal value t*: 15.911948842270723
Optimal control input u*:
[3.23  1.    1.    1.    1.    1.    1.    0.    0.    0.
 0.    0.   -0.  -0.   -0.   -0.   -1.   -1.   -1.  -1.23]

Finally, we visualize the result by extracting optimal u* from cvxpy solution and simulating system
dynamics with optimal u* with the assistance of library matplotlib

# Extract optimal u* from CVXPY solution
u_star = u.value.ravel()
# Simulate system dynamics with optimal u*
A = np.array([[1, 1], [0, 0.95]], dtype=float)
b = np.array([0, 0.1])
n = 2
x = np.zeros((n, N+1))
for t in range(N):
    x[:, t+1] = A @ x[:, t] + b * u_star[t]

# Plot the input signal at time t
plt.title('Minimum-fuel-actuator-signal.')
plt.step(np.arange(N), np.ravel(u.value))
plt.xlabel('Time-t')
plt.ylabel('Actuator-u(t)')

# Plot the position and velocity
plt.subplot(2,1,1)
plt.title('The-position-of-the-vehicle-at-time-t.')
plt.plot(np.arange(N+1), x[0,:])
plt.ylabel('Position-$x_1(t)$')
plt.xlabel('Time-t')

plt.subplot(2,1,2)
plt.title('The-velocity-of-the-vehicle-at-time-t.')
plt.plot(np.arange(N+1), x[1,:])
plt.ylabel('Velocity-$x_2(t)$')
plt.xlabel('Time-t')

plt.tight_layout()
plt.show()

```

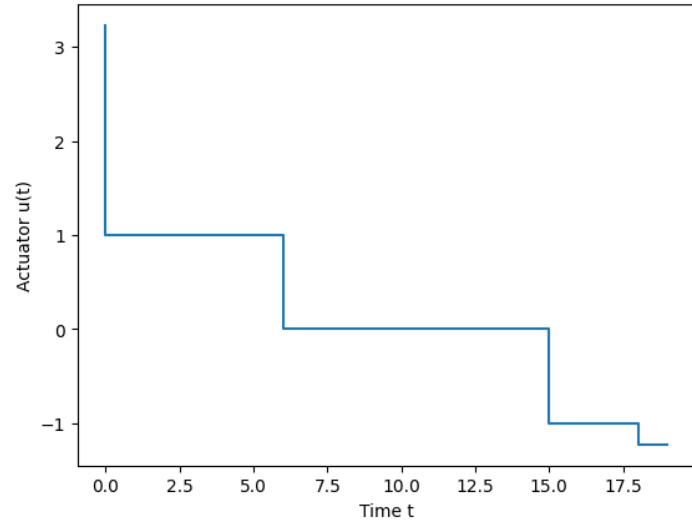


Figure 1: Minimum fuel actuator signal.

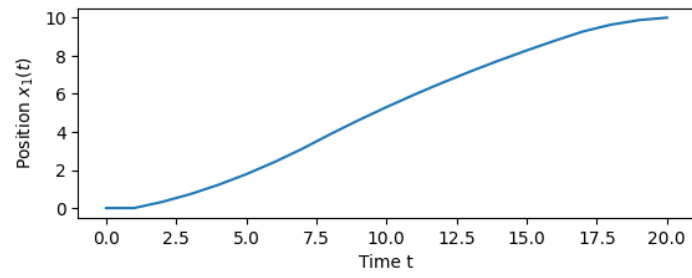


Figure 2: The position of the vehicle at time t .

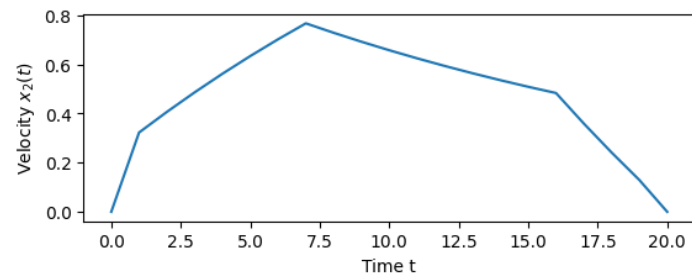


Figure 3: The velocity of the vehicle at time t .

Problem 2

Consider the primal linear programming problem:

$$\begin{aligned} & \text{minimize } z = c^T x \\ & \text{subject to } Ax = b \\ & \quad x \geq 0 \end{aligned}$$

Assume that this problem and its dual are both feasible. Let x^* be an optimal solution to the primal and let y^* be an optimal solution to the dual. For each of the following changes, describe what effect they have on x^* and y^* , if any. These changes should be considered individually, they are not cumulative.

- a) The vector c is multiplied by λ , where $\lambda > 0$.
- b) The k^{th} equality constraint is multiplied by λ .
- c) The i^{th} equality constraint is modified by adding to it λ times the k^{th} equality constraint.
- d) The *right-hand* side b is multiplied by λ .

Solution.

Primal:

$$\begin{aligned} & \text{minimize } z = c^T x \\ & \text{subject to } Ax = b \\ & \quad x \geq 0 \end{aligned}$$

We have x^* as an optimal solution to the primal. p^* is an optimal value to the primal ($p^* = c^T x^*$).

Dual:

$$\begin{aligned} & \text{maximize } z = -b^T y \\ & \text{subject to } A^T y + c \geq 0 \end{aligned}$$

We have y^* as an optimal solution to the dual. d^* is an optimal value to the dual ($d^* = -b^T y^*$). Therefore

$$y^* = \frac{c^T x^*}{-b^T}$$

- a) The vector c is multiplied by λ , where $\lambda > 0$.

Primal:

$$\begin{aligned} & \text{minimize } z = \lambda c^T x \\ & \text{subject to } Ax = b, x \geq 0 \end{aligned}$$

Dual:

$$\begin{aligned} & \text{maximize } -b^T y \\ & \text{subject to } A^T y + \lambda c \geq 0 \end{aligned}$$

When the vector c is multiplied by λ , where $\lambda > 0$. The feasible set of primal doesn't change.

So, we can conclude that x^* is not changed.

However, the optimal value p^* changes. Therefore, we have $p^* = \lambda c^T x^*$.

So, based on the Strong duality theorem:

$$\lambda c^T x^* = -b^T y^* \iff \text{new } y^* = \frac{\lambda c^T x^*}{-b^T} \neq \text{first } y^* = \frac{c^T x^*}{-b^T}$$

Thus, y^* is changed.

b) The k^{th} equality constraint is multiplied by λ .

Primal:

$$\begin{aligned} & \text{minimize } z = c^T x \\ & \text{subject to } \lambda A_k x_k = \lambda b_k (\text{the } k^{th} \text{ constraint}) \iff A_k x_k = b_k \\ & x \geq 0 \end{aligned}$$

Multiplying the k^{th} equality constraint by λ does not change the primal feasible region. So, x^* and p^* **are not change**. However, the change in the primal constraint alters the optimal value of the dual.

Let $w_k = \lambda y_k$ and $w_i = y_i$ for $i \neq k$. We have the k^{th} of the dual $\lambda A_k^T y_k + c_k = A_k^T w_k + c_k$, the optimal value of dual $d^* = -b^T w^*$ (w^* is the new optimal solution of the new dual).

Based on Strong Duality Theorem, we have:

$$w^* = \frac{c^T x^*}{-b^T}.$$

Thus, y^* is changed.

c) The i^{th} equality constraint is modified by adding to it λ times the k^{th} equality constraint.

The new constraint obtained by adding i^{th} equality constraint to λ times the k^{th} equality constraint is equivalent to the old one, so the feasible set does not change.

Therefore, similar to b), x^* and p^* are not changed.

Let $A'x = b'$ be the new constraint of the primal. So, we have:

Dual:

$$\begin{aligned} & \text{maximize } -b'^T y \\ & \text{subject to } A'^T y + c \geq 0 \end{aligned}$$

Based on Strong Duality Theorem, we have:

$$y^* = \frac{c^T x^*}{-b'^T}.$$

Thus, y^* is changed.

d) The *right-hand* side b is multiplied by λ .

Primal:

$$\begin{aligned} & \text{minimize } z = c^T x \\ & \text{subject to } Ax = b\lambda \Rightarrow x = (A^{-1}b)\lambda \\ & x \geq 0 \end{aligned}$$

Dual:

$$\begin{aligned} & \text{maximize } -\lambda b^T y \\ & \text{subject to } A^T y + c \geq 0 \end{aligned}$$

When the *right-hand* side b is multiplied by λ , we have x^* changes. Let h^* be the new optimal solution of the primal, $h^* = (A^{-1}b)\lambda = \lambda x^*$.

So, $p^* = c^T \lambda.x^*$.

Based on Strong Duality Theorem, we have

$$y^* = \frac{c^T \lambda.x^*}{-\lambda b^T} = \frac{c^T x^*}{-b^T}$$

Thus, y^* is not changed.

Problem 3

Consider the following optimization problem in x :

$$\begin{aligned} & \text{minimize } c^T x \\ & \text{subject to } \|Ax + b\|_1 \leq 1 \end{aligned}$$

where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$.

- Formulate this problem as an LP in inequality form
- Derive the dual LP, and show that it is equivalent to the problem

$$\begin{aligned} & \text{maximize } b^T z - \|z\|_\infty \\ & \text{subject to } A^T z + c = 0. \end{aligned} \quad (*)$$

What is the relation between the optimal z and the optimal variables in the dual LP?

- Give a direct argument (i.e., not quoting any results from LP duality) that when never x is primal feasible (i.e., $\|Ax + b\|_1 \leq 1$) and z is dual feasible (i.e., $A^T z + c = 0$), we have

$$c^T x \geq b^T z - \|z\|_\infty$$

Solution.

a)

The problem can be formulated as the LP

$$\begin{aligned} & \text{minimize } c^T x \\ & \text{subject to } \begin{bmatrix} A & -I \\ -A & -I \\ 0 & 1^T \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \leq \begin{bmatrix} -1 \\ b \\ b \end{bmatrix}, \end{aligned}$$

with variables $x \in \mathbb{R}^n$, $y \in \mathbb{R}^m$.

b)

The dual LP is

$$\begin{aligned} & \text{maximize } b^T u - b^T v - w \\ & \text{subject to } A^T u - A^T v + c = 0 \\ & \quad -u - v + w1 = 0 \\ & \quad u \geq 0, \quad v \geq 0, \quad w \geq 0, \end{aligned} \quad (**)$$

with variables $u \in \mathbb{R}^m$, $v \in \mathbb{R}^m$, $w \in \mathbb{R}$.

We rewrite the 2 problems (*) and (**) as P1 and P2 respectively:

$$\begin{aligned} \text{P1: maximize } f_1(u, v, w) &= b^\top u - b^\top v - w \\ \text{subject to } A^\top u - A^\top v + c &= 0 \\ -u - v + w\mathbf{1} &= 0 \\ u \geq 0, \quad v \geq 0, \quad w &\geq 0 \end{aligned}$$

$$\begin{aligned} \text{P2: maximize } f_2(z) &= b^\top - \|z\|_\infty \\ \text{subject to } A^\top z + c &= 0 \end{aligned}$$

(m) *Proving $f_1(u^*, v^*, w^*) \leq f_2(z^*)$:*

Given (u^*, v^*, w^*) , we can construct $\hat{z} = u^* - v^*$ (this is not necessarily the optimal z). Since (u^*, v^*, w^*) are feasible in P1, then we have $A^\top u^* - A^\top v^* + c = A^\top \hat{z} - c = 0$. Therefore $\hat{z}$ is feasible in P2. We can also note that

$$|\hat{z}|_j = |u_j^* - v_j^*| \leq |u_j^* + v_j^*| \stackrel{i}{=} u_j^* + v_j^* = w^*, \text{ for all } j \quad (1)$$

where (i) follows because u^* and v^* are non-negative. Since (1) holds for all j , then $\|\hat{z}\|_\infty \leq w^*$. Therefore, we can write

$$f_2(z^*) \stackrel{ii}{\geq} f_2(\hat{z}) = b^\top \hat{z} - \|\hat{z}\|_\infty \geq b^\top (u^* - v^*) - w^* = f_1(u^*, v^*, w^*),$$

where (ii) holds the optimal point by definition of z^* being the optimal point.

(n) *Proving $f_2(z^*) \leq f_1(u^*, v^*, w^*)$:*

Given z^* , we can construct $(\hat{u}, \hat{v}, \hat{w})$ as follows:

$$\begin{aligned} \hat{w} &= \|z^*\|_\infty, \\ \hat{u}_i &= \max\{z_i^*, 0\} + 0.5(\hat{w} - |z_i^*|), \\ \hat{v}_i &= \max\{-z_i^*, 0\} + 0.5(\hat{w} - |z_i^*|). \end{aligned}$$

It can be verified that $\hat{u}_i - \hat{v}_i = z_i^*$ and $\hat{u}_i + \hat{v}_i = \hat{w}_i$. It can also be verified that $\hat{u}$, $\hat{v}$ and $\hat{w}$ satisfy the constraints in P1, namely

$$\begin{aligned} A^\top \hat{u} - A^\top \hat{v} + c &= A^\top z^* + c = 0, \\ -\hat{u} - \hat{v} + \hat{w}\mathbf{1} &= -\hat{w}\mathbf{1} + \hat{w}\mathbf{1} = 0, \\ \hat{u} \geq 0, \quad \hat{v} \geq 0, \quad \hat{w} &\geq 0. \end{aligned}$$

Therefore, $(\hat{u}, \hat{v}, \hat{w})$ is a feasible point. Therefore, we can write

$$f_1(u^*, v^*, w^*) \stackrel{iii}{\geq} f_1(\hat{u}, \hat{v}, \hat{w}) = b^\top (\hat{u} - \hat{v}) - \hat{w} = b^\top z^* - \|z^*\|_\infty = f_2(z^*),$$

where (iii) holds by definition of u^*, v^*, w^* being the optimal point.

Combining (m) and (n) proves that $f_2(z^*) = f_1(u^*, v^*, w^*)$.

To prove the second requirement, i.e. to show how we can construct (u^*, v^*, w^*) from z^* , we note that the point $(\hat{u}, \hat{v}, \hat{w})$ constructed in (n) has an objective value $f_1(\hat{u}, \hat{v}, \hat{w}) = f_2(z^*) = f_1(u^*, v^*, w^*)$. Therefore, $(\hat{u}, \hat{v}, \hat{w})$ is indeed an optimal point, and the construction provided in

(n) is the required construction:

$$\begin{aligned} w^* &= \|z^*\|_\infty, \\ u_i^* &= \max\{z_i^*, 0\} + 0.5(w^* - |z_i^*|), \\ v_i^* &= \max\{-z_i^*, 0\} + 0.5(w^* - |z_i^*|). \end{aligned}$$

c)

We have $\|z\|_\infty = \max_i |z_i| \geq z_i$ for i is the i^{th} element of vector z by the definition of l_∞ -norm. For any vector u, v of the same size m , the dot product of them has the attribute

$$v^T u = \sum_{i=1}^m v_i u_i \leq \|v\|_\infty \|u\|_1$$

by the definition of l_1 -norm.

Therefore, for vector z and vector $Ax + b$

$$z^T(Ax + b) \leq \|z\|_\infty \|Ax + b\|_1 \leq \|z\|_\infty \quad (2)$$

as x is primal feasible.

$$(2) \text{ is equivalent to: } -z^T Ax \geq z^T b - \|z\|_\infty \quad (3).$$

We have:

+) $z^T b = b^T z$ (dot product of 2 vectors b, z)

+) $c = -A^T z$ (z is dual feasible), which is equivalent to $c^T = (-A^T z)^T = -z^T A$

Substitute these to (3), we have proved:

$$c^T x \geq b^T z - \|z\|_\infty$$

Portfolio Optimization

Suppose that an investor wishes to select a set of assets to achieve a good return on the investment while controlling risks of losses. The use of nonlinear models to manage investments began in the 1950s with the pioneering work of Nobel Prize laureate Harry Markowitz, who demonstrated how to reduce the risk of investment by selecting a portfolio of stocks rather than picking individual attractive stocks, and established the trade-off between reward and risk in investment portfolios.

An investment portfolio is defined by the vector $x = (x_1, \dots, x_n)$, where x_j denotes the proportion of the investment to be invested in asset j . Letting μ_j denote the expected rate of return of asset j , the expected rate of return of the portfolio is $\mu^T x$.

Let Σ be the matrix of variances and covariances of the assets' returns. The entry $\Sigma_{j,j}$ is the variance of investment j . A high variance indicates high volatility or high risk; a low variance indicates stability or low risk. The entry $\Sigma_{i,j}$ is the covariance of investments i and j . A positive value of $\Sigma_{i,j}$ indicates assets whose values usually move in the same direction, as often occurs with stocks of companies in the same industry. A negative value indicates assets whose values generally move in opposite directions—a desirable feature in a diversified portfolio. Markowitz defined the risk of the portfolio to be its expected variance $x^T \Sigma x$.

Our optimization problem has two conflicting objectives: to maximize the return $\mu^T x$ and to minimize the risk $x^T \Sigma x$. The relative importance of these objectives will vary depending on the investor's tolerance for risk. We introduce a nonnegative parameter α that reflects the investor's trade-off between risk and return. The objective function in the model will be some combination of the two objectives, parameterized by α , leading to the model:

$$\text{maximize } f(x) = \mu^T x - \alpha x^T \Sigma x$$

subject to constraints:

$$\Sigma x_i = 1, \quad x \geq 0$$

The value of α reflects the investor's aversion to risk. A large value indicates a reluctance to take on risk, with an emphasis on the stability of the investment. A low value indicates a high tolerance for risk with an emphasis on the expected return of the investment.

It can be difficult to choose a sensible value for α . For this reason, it is common to solve this model for a range of values of this parameter. This can reveal how sensitive the solution is to considerations of risk. The solution of the problem for any value of α is called efficient, indicating that there is no other portfolio that has a larger expected return and a smaller variance.

There are, of course, some limitations to our model. First, we do not generally know the theoretical (joint) distribution of the assets' return and will need to estimate the mean and variance from historical data. Denoting the estimate of μ by r and the estimate of Σ by V , the actual problem we solve is:

$$\begin{aligned} &\text{maximize } r^T x - \alpha x^T V x \\ &\text{subject to } \sum_i x_i = 1 \\ &\quad x_i \geq 0 \end{aligned}$$

Second, investors should be aware that past performance is no indicator of future returns. Finally, we note that the matrix V is dense; that is, it has many nonzero elements. As a result, when the number of assets is large, computations involving V can be expensive, making the optimization problem computationally difficult. To illustrate portfolio optimization, consider these problems.

Problem 7.1

How would the formulation to the problem change if a risk-free asset (such as government treasury bills at a fixed rate of return) is also being considered?

Solution.

The objective is to maximize the expected return of a portfolio while minimizing its risk, measured by the variance (or standard deviation) of returns.

The variables in the problem are:

- x_i : The proportion of your total investment allocated to asset i .
- r_i : The expected return of asset i .
- V_{ij} : The covariance between the returns of asset i and asset j .
- α : A parameter representing risk tolerance or aversion.

The problem can be formulated as follows:

$$\begin{aligned} &\text{Maximize: } r^T x - \alpha x^T V x \\ &\text{Subject to } x \geq 0 \quad (\text{Non-negativity constraint}) \\ &\quad \sum_i x_i = 1 \quad (\text{Budget constraint}) \end{aligned}$$

Where:

- r : Vector of expected returns for each asset.
- α : Risk aversion parameter (a measure of how much the investor is willing to trade off return for risk).

If you want to include a risk-free asset, you would need to modify the optimization problem to account for the risk-free rate. The risk-free asset typically has a known return, denoted as r_f .

The modified problem would look like this:

$$\begin{aligned} &\text{Maximize } r^T x + r_f(1 - \sum_i x_i) - \alpha x^T V x \\ &\text{Subject to } x \geq 0 \quad (\text{Non-negativity constraint}) \\ &\quad \sum_i x_i = 1 \quad (\text{Budget constraint}) \end{aligned}$$

In this modified problem, the term $r_f(1 - \sum_i x_i)$ represents the allocation to the risk-free asset. By including this term, you allow the portfolio to invest in the risk-free asset at the risk-free rate. This way, the investor can decide how much to allocate to the risk-free asset in addition to risky assets. The risk-free asset can act as a baseline for risk-free return in the portfolio.

In summary, if you want to include a risk-free asset, you add the risk-free rate r_f and the corresponding allocation to the risk-free asset in the objective function and adjust the budget

constraint accordingly to ensure that the sum of all allocations is equal to 1. This allows you to create a diversified portfolio that combines both risky and risk-free assets.

Problem 7.2

An investor wants to put together a portfolio consisting of the 30 stocks used to determine the Dow Jones industrial average. Use 25 weekly returns ending on the last Friday of last month to find the optimal portfolio. Experiment with different values of the parameter α and plot the corresponding points on the efficient frontier. You will need access to a nonlinear optimization solver. You may need to use a modeling language to formulate the problem for input to the solver.

Solution.

This problem requires a dataset of 25 weekly returns of 30 stocks. The **yfinance** package in Python helps us retrieve historical market data for stocks in a specific time period to get historical price and volume data. We also have to import **matplotlib.pyplot**, **numpy** to process and plot the data, and **cvxpy** to solve optimization problems.

```
import yfinance as yf
import matplotlib.pyplot as plt
import numpy as np
import cvxpy as cp
```

Because the last weekly returns ended on the last Friday of the last month, we specify the period as April 3rd to September 29th, we can use **yfinance** as below:

```
# Download historical stock data by inputting stock symbols and period
stock_data = yf.download("AXP-AMGN-AAPL-BA-CAT-CSCO-CVX-GS-HD-HON-IBM
----INTC-JNJ-KO-JPM-MCD-MMM-MRK-MSFT-NKE-PG-TRV-UNH-CRM-VZ-V-WBA-WMT-DIS-DOW",
    start="2023-4-03", end="2023-09-30")

# Display the data
stock_data = stock_data["Adj-Close"]
stock_data
```


	AAPL	AMGN	AXP	BA	CAT	CRM	CSCO	CVX	DIS	DOW	...
Date											
2023-04-03	165.716690	240.208969	161.952438	215.389999	226.332825	196.490005	51.159763	166.699844	99.760002	53.869221	...
2023-04-04	165.178177	243.245483	160.746048	213.899994	214.103958	197.199997	51.061226	165.807251	99.570000	52.827469	...
2023-04-05	163.313263	248.984406	159.872650	210.000000	210.244278	195.309998	51.061226	166.631195	99.910004	53.197437	...
2023-04-06	164.210831	248.935287	157.639511	211.369995	205.951355	192.550003	50.519276	164.443832	99.970001	53.216908	...
2023-04-10	161.587982	246.557159	159.932190	210.779999	212.213486	191.490005	50.578403	165.071594	100.809998	54.336548	...
...	...	...	...	...	...	...	...	...	...	...	...
2023-09-25	176.080002	266.859985	152.602997	198.779999	272.860077	206.339996	53.096523	168.710007	81.010002	51.090000	...
2023-09-26	171.960007	269.040009	150.501617	195.639999	268.134491	202.490005	52.332047	167.800003	80.050003	50.480000	...
2023-09-27	170.429993	268.980011	149.286621	195.460007	271.308105	202.729996	52.818535	171.039993	79.900002	51.000000	...
2023-09-28	170.690002	270.820007	149.615265	190.429993	274.819946	203.199997	53.493656	170.589996	80.129997	50.930000	...
2023-09-29	171.210007	268.760010	148.579529	191.679993	271.596619	202.779999	53.374516	168.619995	81.050003	51.560001	...

125 rows × 30 columns

Figure 4: Stock data.

Next, we calculate the weekly return. The output columns are the dates of Sundays from April 16th to October 1st.

```
# Calculate weekly return
weekly_returns = stock_data.resample('W').ffill().pct_change()

# Drop the first row (NaN value)
weekly_returns = weekly_returns[1:]

# Display the weekly returns matrix
print(weekly_returns)
```

	AAPL	AMGN	AXP	BA	CAT	CRM
Date						
2023-04-16	0.003340	-0.013106	0.027640	-0.045702	0.069369	0.010906
2023-04-23	-0.001150	-0.024560	0.003431	0.017054	-0.009900	0.022502
2023-04-30	0.028239	-0.016895	-0.014898	0.007945	-0.006674	-0.003316
2023-05-07	0.022926	-0.013389	-0.053861	-0.040816	-0.016956	-0.003932
2023-05-14	-0.004386	-0.015136	-0.030920	0.011899	-0.025803	0.021357
2023-05-21	0.015008	-0.031861	0.033935	0.023867	0.025007	0.042367
2023-05-28	0.001541	-0.029048	0.028048	-0.009052	-0.013875	0.024149
2023-06-04	0.031466	0.005255	0.071992	0.047586	0.070019	-0.011186
2023-06-11	0.000055	0.003164	0.018747	0.018704	0.037065	0.010703
2023-06-18	0.021883	0.049826	0.002854	0.012333	0.043569	-0.016488
2023-06-25	0.009518	-0.011103	-0.024040	-0.066276	-0.044155	-0.007886
2023-07-02	0.039051	-0.022412	0.036473	0.027993	0.049522	0.005569
2023-07-09	-0.016961	-0.015179	-0.021916	0.004452	-0.003536	-0.007905
2023-07-16	0.000052	0.040155	0.021142	0.004809	0.042581	0.094184
2023-07-23	0.006555	0.032494	-0.018282	-0.006194	0.012933	-0.005538
2023-07-30	0.020267	0.006601	-0.028140	0.126959	0.010790	-0.010787
2023-08-06	-0.070674	0.029234	-0.000121	-0.030709	0.061475	-0.048803
2023-08-13	-0.021759	0.079045	-0.000846	0.018845	0.031761	-0.027448
2023-08-20	-0.018561	0.005452	-0.030496	-0.038478	-0.040039	-0.018543
2023-08-27	0.023612	-0.020778	-0.011920	-0.014295	-0.004529	0.022653
2023-09-03	0.060747	0.001287	0.008211	-0.000045	0.050227	0.057574
2023-09-10	-0.059538	0.010596	-0.013720	-0.054297	-0.013869	0.014580
2023-09-17	-0.017791	0.004972	0.040145	-0.014957	-0.011088	-0.045159
...						
2023-09-24	-0.057984	-0.013909				
2023-10-01	0.053030	-0.014906				

[25 rows x 30 columns]

Figure 5: Weekly returns.

After collecting data, we calculate the average weekly return for each stock based on the collected data and compute the variance-covariance matrix using the collected weekly returns data. From that, we formulate optimization problems based on each values of α and defined constraints. For different values of α , the program solves the optimization problem to obtain the corresponding optimal portfolio weights. It calculates the expected return and standard deviation (risk) for each portfolio, then creates a table of optimal portfolios for selected values of α .

```

from prettytable import PrettyTable

# Define the number of stocks (n) and other relevant data
n = len(weekly_returns.columns)
# Calculate expected returns
mu = np.mean(weekly_returns, axis=0).to_numpy()
# Calculate the matrix of variances and covariance
Sigma = weekly_returns.cov().to_numpy()

# Specify the values of alpha (gamma)
gamma_vals = [1, 2, 5, 10, 100]

# Initialize a list to store portfolio allocations
portfolio_allocations = []

# Long-only portfolio optimization
w = cp.Variable(n)
gamma = cp.Parameter(nonneg=True)
ret = mu @ w
risk = cp.quad_form(w, Sigma)
constraints = [cp.sum(w) == 1, w >= 0]

for gamma_val in gamma_vals:
    gamma.value = gamma_val
    prob = cp.Problem(cp.Maximize(ret - gamma * risk), constraints)
    prob.solve()

    # Portfolio allocation, mean return, and portfolio variance
    allocation = w.value
    mean_return = ret.value
    portfolio_variance = risk.value

    # Append the results to the list
    portfolio_allocations.append([gamma_val] + list(allocation)
                                + [mean_return, portfolio_variance])

# Print the portfolio allocations
t = PrettyTable()

t.field_names = [" "] + [f"Stock-{i+1}" for i in range(n)]
                + ["Mean", "Variance"]
for row in portfolio_allocations:

```

```

alpha = row[0]
formatted_row = [f"{val:.2f}" if val != 0 else "-"
                  for val in row[1:-2]]
formatted_row.append(f"{row[-2]:.3f}")
formatted_row.append(f"{row[-1]:.1e}")
t.add_row([alpha] + formatted_row)
print(t)

```

The final stage is to visualize the data. We plot the efficient frontier based on the previous table on a graph where the x-axis represents standard deviation (risk) and the y-axis represents expected return. We spot that the probability of a loss is near 0 for the low-risk value and far above 0 for the high-risk value.

```

# Plot long only trade-off curve.
%matplotlib inline
%config InlineBackend.figure_format = 'svg'

fig = plt.figure()
ax = fig.add_subplot(111)
plt.plot(risk_data, ret_data, "g-")
for i in range(n):
    plt.plot((Sigma[i, i]), mu[i], "ro")
plt.xlabel("Variance")
plt.ylabel("Return")
plt.show()

```

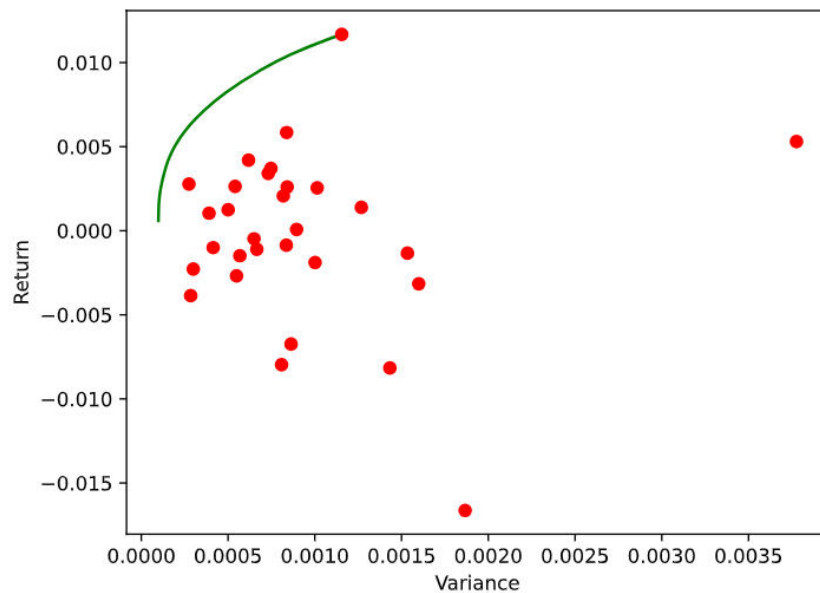


Figure 6: Efficient frontier.